Analysis of `Controller::compute()`

Cascade hover controller for a quadrotor UAV

Computational Modeling for Robotics and HCI — University of Messina

Dynamics and Control of UAVs

Purpose of this document

This document analyses line by line the function `Controller::compute()`, which implements the three-level cascade controller for a quadrotor in hover. For each code block the following are presented: the underlying mathematics, the physical motivation, and the connection with the equations of motion derived in Chapter 5. Guided exercises are proposed at the end.

1 The complete code

Listing 1: Controller::compute() — controller.cpp

```
void Controller::compute(const mjModel* m, mjData* d)
{
    (void)m;
    const double* pos  = d->qpos;        // [x, y, z]
    const double* quat = d->qpos + 3;    // [w, x, y, z]
    const double* vel  = d->qvel;        // linear velocity, world frame
    const double* omg  = d->qvel + 3;    // angular velocity, world frame

    // --- Orientation ---
    double q[4] = {quat[0], quat[1], quat[2], quat[3]};
    qmath::normalize(q);
    double roll, pitch, yaw;
    qmath::to_euler(q, roll, pitch, yaw);

    // angular velocity in body frame: omega_b = R^T * omega_world
    double R[3][3];
    qmath::to_rotmat(q, R);
    double omg_b[3] = {0, 0, 0};
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            omg_b[i] += R[j][i] * omg[j];   // R[j][i] = R^T[i][j]

    // --- Altitude loop ---
    double f_total = DRONE_MASS * (GRAVITY
                   + Kp_z * (z_des - pos[2])
                   + Kd_z * (-vel[2]));
    f_total = std::max(0.0, f_total);

    // --- x/y position loop ---
    const double ax = Kp_xy*(x_des - pos[0]) + Kd_xy*(-vel[0]);
    const double ay = Kp_xy*(y_des - pos[1]) + Kd_xy*(-vel[1]);
    const double theta_des = std::clamp( ax/GRAVITY, -max_tilt, max_tilt
        );
    const double phi_des   = std::clamp(-ay/GRAVITY, -max_tilt, max_tilt
        );
```

```
34
35      // --- Attitude loop ---
36      const double tau_x = Kp_roll  * (phi_des   - roll)                -
            Kd_roll  * omg_b[0];
37      const double tau_y = Kp_pitch * (theta_des - pitch)               -
            Kd_pitch * omg_b[1];
38      const double tau_z = Kp_yaw   * wrap_pi(yaw_des - yaw)            -
            Kd_yaw   * omg_b[2];
39
40      // --- Mixing ---
41      const double l = ARM_LENGTH, c = KM_KF;
42      const double T4 = f_total * 0.25;
43      auto sat = [](double v){ return std::clamp(v, 0.0, MAX_THRUST); };
44      d->ctrl[0] = sat(T4 - tau_z/(4*c) - tau_y/(2*l));  // front CW
45      d->ctrl[1] = sat(T4 + tau_z/(4*c) + tau_x/(2*l));  // left  CCW
46      d->ctrl[2] = sat(T4 - tau_z/(4*c) + tau_y/(2*l));  // back  CW
47      d->ctrl[3] = sat(T4 + tau_z/(4*c) - tau_x/(2*l));  // right CCW
48  }
```

2 MuJoCo interface

The function receives two pointers: `m` (model, constant, holds physical parameters and topology) and `d` (data, updated at every simulation step).

`d->qpos` and `d->qvel` layout for a `freejoint`

Indices	Variable	Meaning
`qpos[0:3]`	x, y, z	position in world frame [m]
`qpos[3:7]`	w, x_q, y_q, z_q	orientation quaternion
`qvel[0:3]`	$\dot{x}, \dot{y}, \dot{z}$	linear velocity, world frame [m/s]
`qvel[3:6]`	$\omega_x, \omega_y, \omega_z$	angular velocity, world frame [rad/s]

The array `ctrl[0..3]` is the only command channel: MuJoCo reads it at the beginning of every `mj_step()` and converts the values into forces and torques via the `<actuator>` section of the XML model file.

3 Block 1 — State reading and quaternion geometry

Listing 2: State reading and normalisation

```
9       double q[4] = {quat[0], quat[1], quat[2], quat[3]};
10      qmath::normalize(q);
11      double roll, pitch, yaw;
12      qmath::to_euler(q, roll, pitch, yaw);
```

Why normalise the quaternion?

A unit quaternion $\boldsymbol{q} = (w, x, y, z)$ with $\|\boldsymbol{q}\| = 1$ represents a pure orientation on $SO(3)$. MuJoCo keeps the norm close to unity during integration, but small numerical errors accumulate; normalisation ensures that the Euler-angle conversion is always numerically stable.

Quaternion → ZYX Euler angles

The ZYX convention (roll–pitch–yaw) decomposes the rotation as:

$$\boldsymbol{R} = R_z(\psi) \cdot R_y(\theta) \cdot R_x(\phi)$$

The three angles are extracted from $\boldsymbol{q} = (w, x, y, z)$ by:

ZYX Euler angles from quaternion

$$\phi = \operatorname{atan2}\big(2(wx + yz),\ 1 - 2(x^2 + y^2)\big) \tag{1}$$

$$\theta = \arcsin\big(2(wy - zx)\big) \tag{2}$$

$$\psi = \operatorname{atan2}\big(2(wz + xy),\ 1 - 2(y^2 + z^2)\big) \tag{3}$$

Gimbal lock singularity

The formula for θ uses arcsin, which has a singularity at $|\sin\theta| = 1$, i.e. $\theta = \pm 90$. At that configuration the system loses one degree of freedom (roll and yaw become indistinguishable). This is why systems performing aggressive manoeuvres use geometric control directly on $SO(3)$ rather than Euler angles.

4 Block 2 — Angular velocity in the body frame

Listing 3: Rotating omega from world frame to body frame

```
14      double R[3][3];
15      qmath::to_rotmat(q, R);
16      double omg_b[3] = {0, 0, 0};
17      for (int i = 0; i < 3; i++)
18          for (int j = 0; j < 3; j++)
19              omg_b[i] += R[j][i] * omg[j];    // R[j][i] = R^T[i][j]
```

Why is this step necessary?

MuJoCo stores the angular velocity in the **world frame**: $\boldsymbol{\omega}^W \in \mathbb{R}^3$. The attitude PD controller needs the components (p, q, r) in the **body frame** to compute the derivative term correctly. The relationship between the two frames is:

$$\boxed{\boldsymbol{\omega}^B = \boldsymbol{R}^T \boldsymbol{\omega}^W}$$

where $\boldsymbol{R}$ is the rotation matrix *from body to world*.

Connection with the implementation

The double loop `omg_b[i] += R[j][i] * omg[j]` computes exactly $\boldsymbol{R}^T\boldsymbol{\omega}^W$: accessing `R[j][i]` instead of `R[i][j]` is equivalent to using the transpose without allocating an extra matrix.

Dimensional check

$\boldsymbol{R} \in \mathbb{R}^{3\times3}$ (orthogonal, $\boldsymbol{R}^T = \boldsymbol{R}^{-1}$), $\boldsymbol{\omega}^W \in \mathbb{R}^3$, so $\boldsymbol{R}^T\boldsymbol{\omega}^W \in \mathbb{R}^3 = \boldsymbol{\omega}^B$ ✓

5 Block 3 — Altitude loop

Listing 4: Altitude PD control

```
21    double f_total = DRONE_MASS * (GRAVITY
22                   + Kp_z * (z_des - pos[2])
23                   + Kd_z * (-vel[2]));
24    f_total = std::max(0.0, f_total);
```

Derivation from the equations of motion

The vertical equation of motion for small angles is:

$$m\ddot{z} = f\cos\phi\cos\theta - mg \approx f - mg$$

Imposing a PD law on the right-hand side:

$$m\ddot{z} = K_{p,z}(z_d - z) + K_{d,z}(0 - \dot{z})$$

Adding gravity compensation (feedforward term):

$$\boxed{f = m\Big(g + K_{p,z}(z_d - z) - K_{d,z}\dot{z}\Big)}$$

This is exactly the code line, with `pos[2]` $= z$ and `vel[2]` $= \dot{z}$.

Steady-state value

At steady state ($\ddot{z} = \dot{z} = 0$, $z = z_d$):

$$f^* = mg = 0.027 \times 9.81 \approx 0.265 \text{ N}$$

Each rotor produces $f^*/4 \approx 0.066$ N, i.e. 44% of its maximum thrust (`MAX_THRUST` = 0.15 N).

The call to `std::max(0.0, f_total)` enforces the **physical constraint**: rotors cannot generate negative thrust.

6 Block 4 — Horizontal position loop (x/y)

Listing 5: Outer-outer loop: horizontal position

```
25    const double ax = Kp_xy*(x_des - pos[0]) + Kd_xy*(-vel[0]);
26    const double ay = Kp_xy*(y_des - pos[1]) + Kd_xy*(-vel[1]);
27    const double theta_des = std::clamp( ax/GRAVITY, -max_tilt, max_tilt
         );
28    const double phi_des   = std::clamp(-ay/GRAVITY, -max_tilt, max_tilt
         );
```

The small-angle linearisation

The full horizontal equations of motion are:

$$m\ddot{x} = f\left(c_\psi s_\theta c_\phi + s_\psi s_\phi\right) \tag{4}$$

$$m\ddot{y} = f\left(s_\psi s_\theta c_\phi - c_\psi s_\phi\right) \tag{5}$$

For $\phi \approx 0$, $\theta \approx 0$, $\psi \approx 0$ and $f \approx mg$:

$$m\ddot{x} \approx f\,\theta \approx mg\,\theta \qquad \Rightarrow \qquad \ddot{x} \approx g\,\theta$$

$$m\ddot{y} \approx -f\,\phi \approx -mg\,\phi \qquad \Rightarrow \qquad \ddot{y} \approx -g\,\phi$$

Inverting, the attitude setpoints needed to achieve a horizontal acceleration (a_x, a_y) are:

$$\boxed{\theta_d = \frac{a_x}{g}, \qquad \phi_d = -\frac{a_y}{g}}$$

where a_x, a_y are the outputs of the position PD:

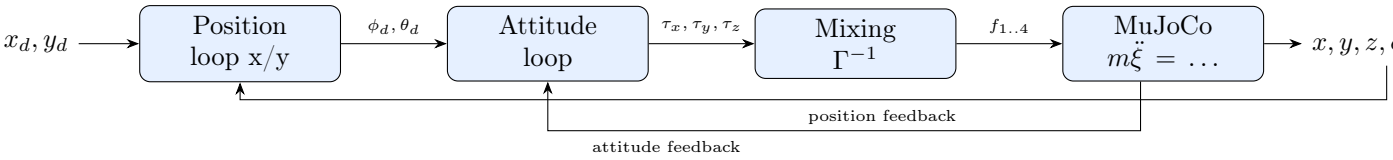
$$a_x = K_{p,xy}(x_d - x) + K_{d,xy}(0 - \dot{x})$$

Tilt limit `max_tilt`

`std::clamp` limits the attitude setpoints to $\pm$`max_tilt` (default: 0.3 rad $\approx$ 17). This is essential for two reasons:

- The linearisation $\ddot{x} \approx g\theta$ breaks down for large angles.
- Large tilts reduce the effective vertical thrust $f\cos\theta$, compromising altitude control.

Cascade diagram: outer-outer → outer → inner



7 Block 5 — Attitude loop (inner loop)

Listing 6: Attitude PD control

```
30    const double tau_x = Kp_roll  * (phi_des   - roll)          -
          Kd_roll  * omg_b[0];
31    const double tau_y = Kp_pitch * (theta_des - pitch)         -
          Kd_pitch * omg_b[1];
32    const double tau_z = Kp_yaw   * wrap_pi(yaw_des - yaw)      -
          Kd_yaw   * omg_b[2];
```

The three torques are computed with the same PD structure:

$$\boxed{\tau_i = K_{p,i}\,e_i - K_{d,i}\,\omega_{b,i}}$$

where e_i is the angular error and $\omega_{b,i}$ is the body-frame angular velocity computed in Block 2.

Why `wrap_pi` for yaw?

The yaw angle ψ lives in $(-\pi, \pi]$. Without `wrap_pi`, the error $e_\psi = \psi_d - \psi$ could equal $+\pi$ or $-\pi$ for the same physical orientation (a full rotation appears to be needed when only a small correction is required). `wrap_pi` maps the error to $(-\pi, \pi)$, ensuring the controller

always takes the shortest path. Example: if $\psi_d = 170$ and $\psi = -170$, the raw error would be 340 (full circle), whereas `wrap_pi` corrects it to -20.

The derivative term uses $\boldsymbol{\omega}^B$ rather than $\dot{\boldsymbol{\eta}}$ (the time derivative of the Euler angles). The two coincide only for small angles; using $\boldsymbol{\omega}^B$ is physically correct in all conditions.

8 Block 6 — Mixing (inversion of Γ)

Listing 7: Inverse rotor force allocation

```
35    const double l = ARM_LENGTH, c = KM_KF;
36    const double T4 = f_total * 0.25;
37    auto sat = [](double v){ return std::clamp(v, 0.0, MAX_THRUST); };
38    d->ctrl[0] = sat(T4 - tau_z/(4*c) - tau_y/(2*l));  // front CW
39    d->ctrl[1] = sat(T4 + tau_z/(4*c) + tau_x/(2*l));  // left  CCW
40    d->ctrl[2] = sat(T4 - tau_z/(4*c) + tau_y/(2*l));  // back  CW
41    d->ctrl[3] = sat(T4 + tau_z/(4*c) - tau_x/(2*l));  // right CCW
```

Derivation of the allocation matrix

From the definition of thrust ($f_i = k_f \Omega_i^2$) and drag torque ($\tau_i = k_m \Omega_i^2$), with $c = k_m/k_f$ and arm l:

$$f_{\text{tot}} = f_1 + f_2 + f_3 + f_4 \tag{6}$$
$$\tau_\phi = l\,(f_2 - f_4) \tag{7}$$
$$\tau_\theta = l\,(f_3 - f_1) \tag{8}$$
$$\tau_\psi = c\,(-f_1 + f_2 - f_3 + f_4) \tag{9}$$

In matrix form $\boldsymbol{u} = \Gamma\,\boldsymbol{f}$:

$$\underbrace{\begin{pmatrix} f \\ \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{pmatrix}}_{\boldsymbol{u}} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & l & 0 & -l \\ -l & 0 & l & 0 \\ -c & c & -c & c \end{bmatrix}}_{\Gamma} \underbrace{\begin{pmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{pmatrix}}_{\boldsymbol{f}}$$

Analytically inverting Γ:

$$\begin{pmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{pmatrix} = \begin{bmatrix} \frac{1}{4} & 0 & -\frac{1}{2l} & -\frac{1}{4c} \\ \frac{1}{4} & \frac{1}{2l} & 0 & \frac{1}{4c} \\ \frac{1}{4} & 0 & \frac{1}{2l} & -\frac{1}{4c} \\ \frac{1}{4} & -\frac{1}{2l} & 0 & \frac{1}{4c} \end{bmatrix} \begin{pmatrix} f \\ \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{pmatrix}$$

The four rows correspond exactly to the four assignments of `ctrl[0..3]` in the code.

The `sat` lambda

In C++14 and later, a local function can be defined inline as a *lambda*:

```
auto sat = [](double v){ return std::clamp(v, 0.0, MAX_THRUST); };
```

It is equivalent to a static function but visible only locally. It enforces the physical constraint: rotor forces must lie in [0, `MAX_THRUST`].

9 Physical parameters and gains

Parameter	Value	Unit	Role
`GRAVITY`	9.81	m/s^2	Gravity feedforward
`DRONE_MASS`	0.027	kg	Total mass (Crazyflie 2.x)
`ARM_LENGTH`	0.046	m	Arm l for mixing
`KM_KF`	0.016	—	Drag/thrust ratio k_m/k_f
`MAX_THRUST`	0.15	N	Per-rotor saturation
`Kp_z`	12.0	N/m	Altitude stiffness
`Kd_z`	5.0	N·s/m	Altitude damping
`Kp_roll`	8×10^{-4}	N·m/rad	Roll stiffness
`Kd_roll`	2×10^{-4}	N·m·s/rad	Roll damping
`Kp_xy`	2.5	N/m	xy position stiffness
`Kd_xy`	2.0	N·s/m	xy position damping
`max_tilt`	0.3	rad (≈ 17)	Max tilt angle

Critical damping rule

For the altitude loop, critical damping is achieved with:

$$K_{d,z} = 2\sqrt{K_{p,z} \cdot m} = 2\sqrt{12.0 \times 0.027} \approx 1.14 \text{ N·s/m}$$

The value used ($K_{d,z} = 5.0$) is larger, so the system is **overdamped** in altitude (slow but non-oscillatory response).

10 Exercises

Exercise 1 — Quaternion analysis (6 pts)

The drone starts with configuration `qpos[3:7] = [1, 0, 0, 0]` (identity quaternion).

a) What are the values of roll, pitch, yaw returned by `qmath::to_euler()`?

b) Substitute the quaternion with $\boldsymbol{q} = (0.966, 0.259, 0, 0)$ (corresponding to roll = 30) and verify formula (1) manually using a calculator.

c) What does `to_euler` return for $\boldsymbol{q} = (0.707, 0, 0.707, 0)$? What is the corresponding pitch angle? Are you close to the singularity? Justify your answer.

Exercise 2 — Rotation of $\boldsymbol{\omega}$ (6 pts)

The drone has angular velocity *in the world frame* $\boldsymbol{\omega}^W = (0,\ 0,\ 1)^T$ rad/s (rotation about the vertical z axis). The body orientation is $\boldsymbol{R} = \mathbf{I}$ (identity).

a) Compute $\boldsymbol{\omega}^B = \boldsymbol{R}^T\boldsymbol{\omega}^W$ by hand. What is the expected result?

b) Repeat with $\boldsymbol{R} = R_x(90)$ (body tilted 90° in roll). What do you observe on the component $r = \omega_{b,2}$?

c) In the code the loop uses `R[j][i]` instead of `R[i][j]`: explain why this is equivalent to multiplying by the transpose without creating a copy of $\boldsymbol{R}$.

Exercise 3 — Altitude loop (8 pts)

a) The drone is at $z = 0.3$ m, zero vertical velocity, setpoint $z_d = 0.5$ m. Compute `f_total`.

b) What would happen if the `GRAVITY` term were removed from the expression? Describe the expected drone behaviour.

c) With $K_{p,z} = 12$, $K_{d,z} = 5$, $m = 0.027$ kg, determine whether the system is underdamped, critically damped, or overdamped using $\zeta = K_{d,z}/(2\sqrt{K_{p,z} \cdot m})$.

d) Propose a value of $K_{d,z}$ that yields critical damping.

Exercise 4 — Linearisation and x/y loop (8 pts)

a) Starting from the full expression for $\ddot{x}$:

$$\ddot{x} = \frac{f}{m}(c_\psi s_\theta c_\phi + s_\psi s_\phi)$$

show analytically that for $\phi, \theta, \psi \to 0$ and $f \to mg$ one obtains $\ddot{x} \approx g\theta$.

b) The drone is at $x = 0.8$ m, $\dot{x} = 0.2$ m/s, setpoint $x_d = 0$, with $K_{p,xy} = 2.5$, $K_{d,xy} = 2.0$. Compute a_x and the corresponding θ_d (in degrees). Does it exceed `max_tilt`? If so, what is the clamped value?

c) Why does ϕ_d have the opposite sign to θ_d relative to the position error? Use a top-view diagram of the drone to support your answer.

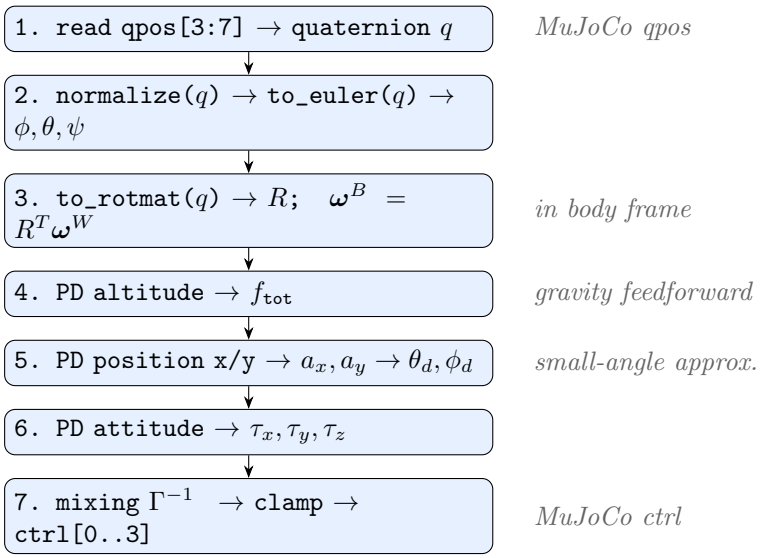
Exercise 5 — Mixing and matrix Γ (12 pts)

a) Write the explicit matrix Γ for the "+" configuration with $l = 0.046$ m and $c = 0.016$.
b) Verify that Γ is invertible by computing $\det(\Gamma)$ (expand along a suitable row or column).
c) Pure hover: $f = mg = 0.265$ N, $\tau_\phi = \tau_\theta = \tau_\psi = 0$. Compute f_1, f_2, f_3, f_4 using the mixing formulas. Is the result physically consistent?
d) Now add a roll torque $\tau_\phi = 5 \times 10^{-5}$ N·m while maintaining hover altitude. Compute the new f_1, f_2, f_3, f_4. Which rotors increase thrust and which decrease it? Is this consistent with the geometry?

Exercise 6 — Open question: external disturbance (10 pts)

A constant wind in the x direction exerts a force $F_d = 0.005$ N on the drone.
a) With the current controller (PD only), does the drone converge to the desired position $x_d = 0$, or does it settle with a steady-state error? Justify analytically using the equations of the x/y loop.
b) Propose a modification to the position loop code to eliminate the steady-state error. Write the formula and indicate where it should be inserted in `compute()`.
c) What precautions must be taken with the integral term in flight-control systems? (*Hint*: consider what happens if the drone is held on the ground and the pilot keeps full stick for 10 seconds.)

Summary: execution flow



Key message

The code contains no physics: the dynamics are entirely delegated to `mj_step()`. `Controller::compute()` implements only the **control law**: it reads the state, computes the algebraic response, and writes the commands. The separation between controller and simulator is the fundamental architectural principle of the project.